

ДЗ_10_АЙКОС

Комкова Полина Дмитриевна БПИ244

Формулировка ДЗ

Есть 100 параллельных источников информации (потоки), порождающих целые числа от **1 до 100**, которые с разной скоростью поступают в **общий буфер**, достаточный для их хранения и обработки. Скорость определяется случайной задержкой от 1 до 7 секунд (функция sleep). Поток, отслеживающий поступление данных (**работает быстро и без задержек**), получив пару любых поступивших чисел, сразу же отправляет их на **суммирование**, создав для этого поток сумматор, который в течение случайного времени (от 3 до 6 секунд) осуществляет суммирование поступивших чисел и направляет их в тот же входной буфер для повторного использования (то есть, для суммирования с новыми поступившими числами или другими промежуточными результатами). То есть, одновременно может быть запущено несколько сумматоров. **Вычисления завершаются, когда будет получен окончательный результат**

Исходный код

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

typedef struct Node {
    int value; // Значение в узле
    struct Node *next; // Указатель на следующий узел
} Node;

Node *head = NULL; // Указатель на голову очереди для pop
Node *tail = NULL; // Указатель на хвост очереди для push
int qsize = 0; // Размер очереди

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // Мьютекс для доступа к
```

очереди

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER; // Условная переменная для диспетчера
```

```
// Счетчики для завершения
```

```
int producers_left = 100; // Сколько производит. еще не положили значение
```

```
int active_summers = 0; // Сколько сумматоров сейчас работают
```

```
int final_result = 0; // Результат
```

```
int done = 0; // Флаг завершения
```

```
void push_back(int v) {
```

```
    Node *n = malloc(sizeof(Node)); // Выделяем память под новый узел
```

```
    if (!n) {
```

```
        perror("malloc");
```

```
        exit(1);
```

```
    }
```

```
    n->value = v; // Записываем узел
```

```
    n->next = NULL; // Указатель на следующий узел
```

```
    if (!tail) { // Если очередь пуста
```

```
        head = tail = n;
```

```
    } else { // Иначе добавляем в хвост
```

```
        tail->next = n;
```

```
        tail = n;
```

```
    }
```

```
    qsize++; // Увеличиваем размер очереди
```

```
}
```

```
int pop_front() {
```

```
    if (!head) {
```

```
        fprintf(stderr, "pop from empty\n");
```

```
        exit(1);
```

```
    }
```

```
    Node *n = head; // Сохраняем указатель на старый узел
```

```
    int v = n->value; // Запоминаем значение
```

```
    head = head->next; // Двигаем голову
```

```
    if (!head) {
```

```
        tail = NULL;
```

```
    }
```

```
    free(n); // Освобождаем память
```

```
    qsize--; // Уменьшаем размер очереди
```

```
    return v; // Возвращаем значение
```

```
}
```

```
typedef struct {
```

```

    int a;
    int b;
    unsigned int seed;
} pair_arg;

void *summer_thread(void *arg) {
    pair_arg *p = (pair_arg*)arg; // Восстанавливаем структуры
    unsigned int seed = p->seed; // Восстанавливаем seed
    int a = p->a;
    int b = p->b;
    free(p);

    int delay = (rand_r(&seed) % 4) + 3; // Случайная задержка от 3 до 6
    sleep(delay); // Пауза

    int s = a + b;

    pthread_mutex_lock(&mutex);
    push_back(s); // Кладем в очередь
    active_summers--; // Уменьшаем количество активных сумматоров
    pthread_cond_broadcast(&cond); // Будим диспетчера
    pthread_mutex_unlock(&mutex);

    printf("Сумматор: %d + %d = %d, размер очереди = %d\n", a, b, s, qsize);
    return NULL;
}

void *dispatcher(void *arg) {
    (void)arg;
    while (1) {
        pthread_mutex_lock(&mutex);
        // Пока нет двух элементов или не все производители отработали
        while (qsize < 2 && !(producers_left == 0 && active_summers == 0 && qsize
== 1)) {
            pthread_cond_wait(&cond, &mutex);
        }

        // Если производители закончили и очередь пуста
        if (producers_left == 0 && active_summers == 0 && qsize == 1) {
            final_result = pop_front(); // Достаем результат
            done = 1;
            pthread_mutex_unlock(&mutex);
            break;
        }

        // Если в очереди хотя бы 2 элемента

```

```

if (qsize >= 2) {
    int x = pop_front();
    int y = pop_front();
    active_summers++; // Увеличиваем количество сумматоров

    pthread_t tid; // Идентификатор потока
    pair_arg *parg = malloc(sizeof(pair_arg)); // Выделяем память
    if (!parg) {
        perror("malloc");
        exit(1);
    }
    parg->a = x;
    parg->b = y;
    parg->seed = (unsigned int)time(NULL) ^ (unsigned int)pthread_self();
    // Создаем поток-сумматор
    if (pthread_create(&tid, NULL, summer_thread, parg) != 0) { // Если не
получилось создать
        perror("pthread_create summer");
        free(parg);
        active_summers--;
    } else { // Успех
        pthread_detach(tid); // Не ждем join
    }
    pthread_mutex_unlock(&mutex);
    printf("Диспетчер: запущен сумматор для (%d,%d); Размер очереди = %d;
активные сумматоры = %d\n", x, y, qsize, active_summers);
    continue;
}
pthread_mutex_unlock(&mutex);
}
return NULL;
}

void *producer(void *arg) {
    int id = *((int*)arg); // Получаем id производителя
    unsigned int seed = (unsigned int)time(NULL) ^ (unsigned int)id;

    int delay = (rand_r(&seed) % 7) + 1; // Задержка от 1 до 7
    sleep(delay);

    pthread_mutex_lock(&mutex);
    push_back(id); // Кладем в очередь
    producers_left--; // Этот производитель закончил
    pthread_cond_broadcast(&cond); // Будим диспетчера
    pthread_mutex_unlock(&mutex);
}

```

```

    printf("Производитель %d: положил %d; размер очереди = %d\n", id, id,
qsize);
    return NULL;
}

int main(void) {
    srand((unsigned)time(NULL));

    const int N = 100; // Количество производителей
    pthread_t prod[N]; // Массив потоков-производителей
    int ids[N]; // Массив id для передачи в потоки

    for (int i = 0; i < N; ++i) {
        ids[i] = i + 1;
        // Запускаем поток-производителя
        if (pthread_create(&prod[i], NULL, producer, &ids[i]) != 0) {
            perror("pthread_create producer");
            return 1;
        }
    }

    // Запускаем диспетчера
    pthread_t disp;
    if (pthread_create(&disp, NULL, dispatcher, NULL) != 0) {
        perror("pthread_create dispatcher");
        return 1;
    }

    // Ждем завершения всех производителей
    for (int i = 0; i < N; ++i) {
        pthread_join(prod[i], NULL);
    }

    pthread_join(disp, NULL);

    printf("\nРезультат = %d\n", final_result);

    while (head) { // Чистим очередь
        pop_front();
    }

    // Освобождаем ресурсы
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    return 0;
}

```

Алгоритм работы

1. Есть 100 **потоков-источников** (у меня: **производители**)
 - Каждый из них ждет случайное время от 1 до 7, а потом генерирует целое число (я сделала числа от 1 до 100 по числу потоков)
ВАЖНО: ответ при запуске из-за этого всегда будет **одинаковый** и равный **5050**.
Во-первых, в условии не дано, что это рандомные числа, во-вторых, я так удостоверилась в том, что работа всегда работает корректно
 - После этого поток добавляет число в **общий буфер (очередь)** и завершает работу
 - Каждый производитель делает это ровно **1 раз**, и я так отслеживаю, сколько из них уже закончили работу через счетчик `prodecers_left`
2. Все данные поступают в **общий буфер (очередь)**, защищенный мьютексом `pthread_mutex_t`, чтобы только один поток мог изменять его в конкретный момент
 - Производители и сумматоры **кладут** числа в очередь, а диспетчер их **извлекает**
3. **Диспетчер** следит за буфером с помощью `pthread_cond_t`:
 - Если в буфере меньше 2х элементов, он ждет сигнала от других потоков
 - Иначе он извлекает два значения и создает поток-сумматор и передает ему числа
4. **Сумматоры** работают параллельно, где каждый ждет случайное время от 3 до 6 секунд, складывают два числа, возвращает сумму обратно в буфер и сообщает диспетчеру об успешном завершении через `cond`
5. **Окончание вычислений** определяется условиями:
 - Все 100 производителей уже завершили работу
 - Нет активных сумматоров
 - В буфере находится ровно 1 число - **результат**

Пример использования

```

polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_10
Сумматор: 369 + 237 = 606, размер очереди = 1
Сумматор: 156 + 131 = 287, размер очереди = 2
Диспетчер: запущен сумматор для (606,287); Размер очереди = 0; активные сумматоры = 12
Сумматор: 316 + 342 = 658, размер очереди = 1
Сумматор: 194 + 284 = 478, размер очереди = 2
Диспетчер: запущен сумматор для (658,478); Размер очереди = 0; активные сумматоры = 11
Сумматор: 288 + 174 = 462, размер очереди = 1
Сумматор: 40 + 33 = 73, размер очереди = 2
Диспетчер: запущен сумматор для (462,73); Размер очереди = 0; активные сумматоры = 10
Сумматор: 326 + 216 = 542, размер очереди = 1
Сумматор: 50 + 112 = 162, размер очереди = 2
Диспетчер: запущен сумматор для (542,162); Размер очереди = 0; активные сумматоры = 9
Сумматор: 82 + 174 = 256, размер очереди = 1
Сумматор: 89 + 228 = 317, размер очереди = 2
Диспетчер: запущен сумматор для (256,317); Размер очереди = 0; активные сумматоры = 8
Сумматор: 232 + 146 = 378, размер очереди = 1
Сумматор: 216 + 227 = 443, размер очереди = 2
Диспетчер: запущен сумматор для (378,443); Размер очереди = 0; активные сумматоры = 7
Сумматор: 239 + 149 = 388, размер очереди = 1
Сумматор: 606 + 287 = 893, размер очереди = 2
Диспетчер: запущен сумматор для (388,893); Размер очереди = 0; активные сумматоры = 6
Сумматор: 658 + 478 = 1136, размер очереди = 1
Сумматор: 462 + 73 = 535, размер очереди = 2
Диспетчер: запущен сумматор для (1136,535); Размер очереди = 0; активные сумматоры = 5
Сумматор: 542 + 162 = 704, размер очереди = 1
Сумматор: 256 + 317 = 573, размер очереди = 2
Диспетчер: запущен сумматор для (704,573); Размер очереди = 0; активные сумматоры = 4
Сумматор: 1136 + 535 = 1671, размер очереди = 1
Сумматор: 704 + 573 = 1277, размер очереди = 2
Диспетчер: запущен сумматор для (1671,1277); Размер очереди = 0; активные сумматоры = 3
Сумматор: 378 + 443 = 821, размер очереди = 1
Сумматор: 388 + 893 = 1281, размер очереди = 2
Диспетчер: запущен сумматор для (821,1281); Размер очереди = 0; активные сумматоры = 2
Сумматор: 1671 + 1277 = 2948, размер очереди = 1
Сумматор: 821 + 1281 = 2102, размер очереди = 2
Диспетчер: запущен сумматор для (2948,2102); Размер очереди = 0; активные сумматоры = 1
Сумматор: 2948 + 2102 = 5050, размер очереди = 1

Результат = 5050
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_10$

```

Структура файла

```

HW_10/
├── HW_10.c           # Программа
└── README.md        # Описание программы

```

Компиляция

- *Запускаем 1 терминал*

```
gcc HW_10.c -o main -lpthread
```